Comparison between BHCVRP and OR-Tools

Ananda de la Caridad Morales Morales [0009-0003-9208-8029], Dayana Vázquez Rodríguez [0009-0004-2381-2180], Dr. C. Isis Torres Pérez [0000-0002-1079-8285] and Dr. C. Alejandro Rosete [0000-0002-4579-3556]

[1] Technological University of Havana "José Antonio Echeverría", Havana, Cuba 19390
nanda.snow.black@gmail.com

Abstract. The Vehicle Routing Problem (VRP) is a significant combinatorial optimization problem involving a fleet of vehicles serving geographically dispersed customers while minimizing costs and meeting constraints. As new practical situations arise, the number of these problems and their variants continues to increase. One method for solving VRP is construction heuristics, which aims to quickly find a near-optimal solution. However, most heuristics are limited to the basic variant known as CVRP, and few libraries implements these methods. For example, BHCVRP addresses this by solving four types of VRP using nine construction heuristics. The motivation for this research arose from the growing Python community that seeks solutions to combinatorial optimization problems such as VRP. For this purpose, it is necessary to know tools developed in this programming language. Among the most prominent implements in this context is OR-Tools. Therefore, the objective of this paper is to conduct a comparative analysis of some heuristics from BHCVRP and OR-Tools, with a center on the quality of solutions generated and the execution time. It is important to note that the experiments in this paper focus solely on applying construction heuristics and the most well-known VRP variants in the literature that are common in both software components. To do this, five instances of the CVRP variant and five instances of the HFVRP, as well as the heuristics Savings Algorithm and the Parallel Insertion Heuristic of Christofides, Mingozzi and Toth, have been considered.

Keywords: BHCVRP, construction heuristics, OR-Tools, Vehicle Routing Problem.

1 Introduction

The Vehicle Routing Problem (VRP) involves creating efficient routes for a fleet of vehicles to serve a group of customers while meeting specific constraints. This problem is commonly used in supply chain management to deliver goods and services. Various versions of VRP are designed based on factors such as the nature of the transported goods, customer characteristics, and vehicle specifications [1]. According to [2], VRP was first introduced in 1959 as a generalized form of the Traveling Salesman Problem (TSP). In 1964, the Savings Algorithm, the first effective algorithm for solving VRP, was proposed [3]. Subsequently, research and work in the field of VRP increased, leading to the development of models that incorporate real-life features and the search for efficient algorithms to solve these problems [4]. Basic models are proposed to solve practical situations, which can be extended to different logistics and transportation scenarios. VRP has practical applications in employee transportation, bus routes, solid waste collection, newspaper and mail distribution, and others. These everyday situations can be modeled with the characteristics of different existing VRP variants [5]. Some well-known variants of VRP include the Capacitated Vehicle Routing Problem (CVRP) [6], Multiple Depot Vehicle Routing Problem (MDVRP) [7], Heterogeneous Fleet Vehicle Routing Problem (HFVRP) [8], Truck and Trailer Routing Problem (TTRP) [9], and more.

Vehicle Routing Problems (VRPs) are often difficult to solve, especially when dealing with a large number of customers. This is because VRPs are classified as NP-hard problems, which are known for being very challenging to solve [1]. Sometimes, it may be necessary to compromise on optimality and instead focus on constructing a good solution, even if it is not guaranteed the best possible solution. To address this, heuristic algorithms, including heuristics and metaheuristics, are used as alternative solutions [10, 11]. Heuristics are designed to guide the search process and recommend the best course of action when faced with multiple options [12, 13]. They are procedures aimed at providing solutions to problems with good performance in terms of solution quality and resources used. Heuristics offer simpler methods that deliver satisfactory solutions using problem-specific algorithms. They can either provide the final solution to a problem or be utilized within the context of metaheuristics [4, 13]. Heuristics are generally categorized into two groups: construction heuristics and improvement heuristics [11, 14].

The construction heuristics are responsible for creating a solution step by step, while improvement heuristics work on existing solutions to achieve enhancements [14]. Commonly used construction heuristics for solving VRPs include the Savings Algorithm, Sweep Algorithm, Parallel Insertion Heuristic of Christofides, Mingozzi and Toth, the Sequential Insertion Heuristic of Mole & Jameson, and others [15]. Due to the necessity of solving combinatorial optimization problems using such algorithms and to promote the reusability of their codes, it is beneficial to organize them within software components. Currently, few libraries implement heuristic methods for solving VRPs and other optimization problems. Some software components meeting these criteria include BiCIAM [16], METSlib [17], MALLBA [18], VRPH [19], MOMHLib++ [20], BHCVRP [21], OR-Tools [22], JSprit [23], VROOM [24], and others.

Among the software components mentioned, the following are used for solving VRP: BiCIAM, VRPH, BHCVRP libraries, OR-Tools, JSprit, and VROOM tools. BHCVRP is the only one that exclusively allows for solving through construction heuristics. The literature shows a lack of heuristic libraries for solving combinatorial optimization problems, as most development focuses on metaheuristic algorithms. There are few adaptations of heuristics to the different VRP variants, and the specific characteristics of each problem are often not considered in the original versions. Furthermore, most existing software components for solving these problems are developed in C++ and

Java. Given the growth of the Python community in Artificial Intelligence and optimization problems, it would be valuable to explore how to solve VRP with heuristics using this language [25, 26].

Considering these antecedents, the aim of this study is to compare BHCVRP and OR-Tools for solving VRPs using the Savings Algorithm and Parallel Insertion Heuristic of Christofides, Mingozzi, and Toth. Particularly, the study is centered on the CVRP and HFVRP variants with the Euclidean distance.

The paper is structured as follows. Section 2 discusses the general characteristics of the CVRP and HFVRP variants, construction heuristics, and provides a comprehensive review of the subjects. Section 3 covers the essential information related to BHCVRP and OR-Tools components, as well as their configurations with the selected construction heuristics. Section 4 presents and discusses the experimental results. Finally, Section 5 includes the conclusions and outlines future work.

2 Heuristics to solving VRPs

The basic models developed for VRP are used to solve real-world scenarios in the logistics and transportation industry. Some practical applications of VRP include employee transportation, bus routes, solid waste collection, and newspaper and mail distribution, among others. These everyday scenarios can be represented using various existing VRP variants or a combination of these, while taking into account their specific characteristics [1]. This paper deals specifically with the CVRP and HFVRP variants, since they are common in the two software components that will be analyzed. In addition, they serve as a basis for more complex variants such as [27].

The Capacitated Vehicle Routing Problem (CVRP) is a version of the VRP that includes constraints related to vehicle capacity. Each customer has a specific demand to meet, and the total demand of the customers on each route cannot exceed the capacity of the vehicle used on that route. This problem involves a set of identical vehicles located in a central depot tasked with satisfying customer demand while adhering to capacity constraints [12].

On the other hand, the Heterogeneous Fleet Vehicle Routing Problem (HFVRP) involves vehicles with different characteristics and/or capacities. The heterogeneous capacity is taken into account when determining which routes to build, as a larger vehicle can handle a longer route or a route with a higher concentration of demand. The number of vehicles of each type is limited, and there is a fixed cost or cost per unit of distance traveled associated with such vehicles [8].

In addressing these types of issues, it is common practice to utilize heuristic algorithms to find solutions. Heuristics are widely used as an alternative to ensure the best possible results in VRPs. They are further classified into construction heuristics and improvement heuristics [10, 11]. This paper focuses on construction heuristics as a solution method to be explored in the BHVRP and OR-Tools.

2.1 Construction heuristics

The construction heuristics for VRPs can be categorized into four groups according to [28]: saving-based, cost-based, route first – cluster second, and cluster first – route second. Saving-based heuristics involve substituting nodes on the route with others that are not yet there, maintaining a constant exchange process as long as the constraints are met. Well-known algorithms in this category include the Savings Algorithm with its sequential and parallel versions, as well as the variant based on Matching [15]. Cost-based heuristics are used to generate a set of initial routes, with solutions constructed by constant insertions of nodes. Popular heuristics in this category include Nearest Neighbor, Mole & Jameson's Sequential Insertion (MJ), Parallel Insertion Heuristic of Christofides, Mingozzi and Toth (CMT), and Kilby's Insertion [15]. Route first–cluster second heuristics determine the route to visit all customers and then partition it into several feasible routes, with algorithms like Petal Algorithms exhibiting this behavior [15]. Lastly, cluster first–route second heuristics aim to generate a group of customers belonging to the same route in the final solution, creating a route for each group that visits all its clients by determining the order in which they will be visited. Well-known algorithms in this category include the Sweep Algorithm, the Gillet-Miller Algorithm, and the Generalized Assignment Heuristic of Fisher and Jaikumar [15]. In this paper, the focus is on the construction heuristics: Savings Algorithm and CMT. These are the construction heuristics that both BHCVRP and OR-Tools have in common. Additionally, they stand out in the ranking for the quality of solutions obtained for different VRPs in both software components.
The Savings Algorithm is the first effective algorithm for solving a VRP. It involves merging two distinct routes (0, …, i,0) and (0, j, …, 0), to create a new route (0, …, i, j, …, 0) within a solution [3]. An example of this is shown in the below **Fig. 1**:

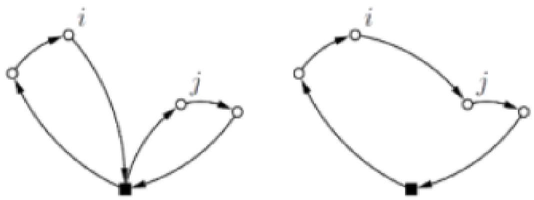

Fig. 1. Example of two routes that can be combined [21].
The savings in distance that result from such a union are calculated in accordance with $Sij = Ci0 + C0j - Cij$(1):

$$Sij = Ci0 + C0j - Cij \quad (1)$$

The new solution eliminates the use of arcs (i,0) and (0, j) while incorporating arc (i, j). The algorithm begins with an initial solution and identifies combinations that result in the greatest savings, provided that the problem's constraints are satisfied. There are three versions of the algorithm: a parallel one that works on all routes simultaneously, a sequential one that builds the routes one by one, and a matching-based one considering the impact of the joint to be made on potential future joints [15, 21].

On the other hand, the algorithm proposed by Christofides, Mingozzi and Toth (CMT) is a cost-based insertion heuristic that operates in two phases. In the first phase, the number of routes to use is determined, along with a client to initialize each route [29]. To achieve this, a sequential insertion algorithm is employed to generate compact routes. To initiate the *kth* path, is identified the client V_k that has not yet been visited. The cost of inserting client w into the path containing V_k is calculated by the formula set out in $\delta_{w,V_k} = C_{0w} + \lambda C_{w,V_k}$ (2):

$$\delta_{w,V_k} = C_{0w} + \lambda C_{w,V_k} \tag{2}$$

If the insertion is not feasible, the value of δ_{w,V_k} is set to ∞. The path is assigned customers in descending order of δ until no further feasible insertions are possible. In the second phase, these routes are created, and the rest of the clients are inserted into them. In order to find the best order for adding customers to a route, calculate the cost difference between adding the customer to that route and the next best alternative. The bigger difference, the more important it is to add the customer to that route first [15, 21].

3 BHCVRP and OR-Tools to solve VRP

To solve Vehicle Routing Problems, it is crucial to use a library that incorporates heuristic methods to ensure their reusability. The majority of research has focused on creating libraries with metaheuristic algorithms, as these are generally applicable to any optimization problem. For this paper, BHCVRP and OR-Tools were selected as the software components, as they offer efficient heuristic solutions in Python for solving VRPs.

3.1 BHCVRP

BHCVRP (the Spanish acronym for Library of Construction Heuristics for Vehicle Routing Problems) is a class library that implements construction heuristics for solving VRPs. This library was developed in [21] in the Java programming language with an n-tier architecture with a reuse-based approach and contains the adaptation of seven classical heuristics for the CVRP, MDVRP, HFVRP, and TTRP variants. The implemented heuristics are the Sweep Algorithm, two versions of the Savings Algorithm (Sequential and Parallel), the Nearest Neighbor with Restricted List Candidates heuristic (NN with RLC), the Mole & Jameson Insertion Heuristic (MJ), the CMT Heuristic, and a randomized method [21]. In its latest version, which is still under development, the following construction heuristics have been incorporated: Kilby Insertion and Matching-based Savings Algorithm. In addition, an implementation in the Python language is proposed that solves some of the previous deficiencies such as the treatment of own exceptions, the flexibility and reuse to model the heuristics using the Template Method design pattern, the incorporation of BHAVRP [30] to work on the assignment of clients to deposits in the MDVRP variant and the use of post-optimization methods. Based on the results obtained in previous academic experiments, it is determined that for the CVRP variant, the algorithm with the best performance in terms of quality of solutions obtained is the parallel version of the Savings Algorithm, while for HFVRP it was the sequential version of the Savings Algorithm.

3.2 OR-Tools

Google OR-Tools is a free, open-source optimization library developed by Google, designed to solve a wide range of combinatorial optimization problems, including VRPs. Written primarily in C++, OR-Tools also provides interfaces to several programming languages such as Python, C#, and Java. This tool offers a variety of algorithms for linear programming problems, integer programming, and mixed integer linear programming, among others. It is recognized for its power and flexibility in solving optimization problems in industries such as logistics and transportation, thanks to its efficiency and ability to handle large-scale problems. It stands out for its high performance, allowing it to solve complex problems in a reasonable time, and for its diversity of algorithms that make it possible for users to choose the best option for their specific task [31].

OR-Tools provides detailed documentation and examples of use and has an active community of users who share knowledge and experiences. The library solves the TSP and VRP variants as CVRP and HFVRP, in addition to other variants with additional constraints. It also uses the heuristics Savings (SV), Path Cheapest Arc (PCA), Christofides (CH), Local Cheapest Arc (LCA), Evaluator Strategy (ES), First Unbound Min Value (FUMV), and metaheuristics [31]. It is important to note that the only heuristics that belong to the literature and match those implemented in BHCVRP are the CH and SV algorithms.

4 Results and discussion

In this section, an experiment is developed to determine which algorithm is better at solving the CVRP and HFVRP variants. For this purpose, the heuristics Savings Algorithm and Parallel Insertion Heuristics of Christofides, Mingozzi, and Toth (CMT), both implemented in the software components BHCVRP and OR-Tools, are taken into account. These tools are chosen since both are used for theses of the Faculty of Computer Engineering of the Technological University of Havana "José Antonio Echeverría" (CUJAE) and are exponents to solve VRPs from the Python programming language. For the experiment execution, it selected instances from the literature for the two VRP variants. **Table 1** and **Table 2** show the cases used in the CVRP and HFVRP variants, respectively.

For this purpose, the CVRP variant has selected instances from the author Cordeau available in [32], with slight transformations. For each instance of [32] the clients, the number of vehicles and their capacities are kept with their original data, only adapted to work with a single depot. However, for the HFVRP variant, no test instances are available in the literature, and it is decided to use the cases selected for the CVRP variant. A total of ten test instances are used for experimentation, but only the five most representative are presented below.

Table 1. Description of CVRP instances.

Id	Instance	Total customers	Total vehicles	Capacity of the vehicles
1	CVRP_1	50	5	200
2	CVRP_4	75	9	250
3	CVRP_p3	75	3	480
4	CVRP_p5	100	5	400
5	CVRP_p14	360	5	500

Table 2. Description of HFVRP instances.

Original data				Vehicle fleet	
Id	Instance	Total customers	Total vehicles	Total	Capacity
1	HFVRP_1	50	5	1	200
				1	400
				1	100
				1	250
				1	50
2	HFVRP_4	75	9	2	250
				2	100
				2	455
				1	90
				1	50
				1	500
3	HFVRP_p3	75	3	2	470
				1	500
4	HFVRP_p5	100	5	1	400
				1	100
				1	200
				2	650
5	HFVRP_p14	360	5	3	500
				1	400
				1	600

When comparing solutions, consider the cost in Euclidean distance, the number of routes obtained for the best solutions (shown in parentheses), and the execution time in seconds. BHCVRP utilizes the Savings Algorithm in three versions: Save Parallel (SP), Save Sequential (SS), and Matching-based Saving Algorithm (MS), as well as CMT. However, OR-Tools uses Savings (SV) and Christofides (CH). The results are presented in **Table 3**, **Table 4**, **Table 5**, and **Table 6** below. The best results are highlighted in bold type. Each experiment involves 20 executions of the heuristics for each instance with 20 evaluations of the objective function.

Table 3. Summary of the results obtained by BHCVRP and OR-Tools with the Savings Algorithm in the CVRP variant with Euclidean distance.

Instance	BHCVRP						OR-Tools	
	SP		SS		MS		SV	
	Cost in distance	Execution time (s)	Cost in distance	Execution time (s)	Cost in distance	Execution time (s)	Cost in distance	Execution time (s)
CVRP_1	561.46 (4)	1.25	**558.89 (4)**	0.54	953.83 (5)	0.67	2215.0 (4)	**0.07**
CVRP_4	**733.2 (6)**	3.18	787.61 (6)	1.26	1168.18 (6)	2.13	3979.0 (6)	**0.27**
CVRP_p3	**669.99 (3)**	2.94	674.88 (3)	1.22	1133.91 (3)	2.28	4203.0 (3)	**0.16**
CVRP_p5	**785.84 (4)**	6.50	855.49 (4)	2.74	1679.47 (4)	4.26	4651.0 (4)	**0.64**
CVRP_p14	**5386.65 (4)**	309.67	6134.72 (4)	94.81	33783.06 (5)	180.04	10097.0 (4)	**6.26**

Table 4. Summary of the results obtained by BHCVRP and OR-Tools with the CMT heuristic in the CVRP variant with Euclidean distance.

Instance	BHCVRP		OR-Tools	
	CMT		CH	
	Cost in distance	Execution time (s)	Cost in distance	Execution time (s)
CVRP_1	**598.96 (4)**	0.5086	2185.0 (4)	**0.13**
CVRP_4	**806.83 (6)**	1.3093	3966.0 (6)	**0.20**
CVRP_p3	**800.91 (3)**	1.7931	4144.0 (4)	**0.31**
CVRP_p5	**1005.45 (4)**	3.3230	4542.0 (4)	**0.57**
CVRP_p14	**16421.59 (4)**	31.3677	9659.0 (4)	**9.29**

From these results, it can be observed in **Table 3** that in four instances the best cost-effective solutions were obtained with the SP heuristic, while the remaining instance was with the SS heuristic. However, the SV heuristic obtained better execution times. In addition, **Table 4** shows that the CMT heuristic obtains better quality solutions, while CH presents better execution times.

Table 5. Summary of the results obtained by BHCVRP and OR-Tools with the Savings Algorithm in the HFVRP variant with Euclidean distance.

Instance	BHCVRP						OR-Tools	
	SP		SS		MS		SV	
	Cost in distance	Execution time (s)	Cost in distance	Execution time (s)	Cost in distance	Execution time (s)	Cost in distance	Execution time (s)
HFVRP_1	**474.33 (2)**	1.09	550.34 (4)	0.482	839.36 (3)	1.75	2507.0 (3)	**0.18**
HFVRP_4	851.96 (7)	1.18	**802.7 (6)**	1.44	1133.9 (3)	7.10	3766.0 (4)	**0.35**
HFVRP_p3	**653.83 (3)**	4.19	666.31 (3)	1.36	1170.71 (3)	3.73	4083.0 (3)	**0.27**
HFVRP_p5	982.69 (4)	13.60	**768.28 (3)**	2.66	1613.45 (3)	9.88	1558.0 (3)	**0.68**
HFVRP_p14	7245.90 (4)	41.33	**6182.04 (4)**	24.61	35223.4 (4)	425.69	9628.0 (4)	**9.66**

Table 6. Summary of the results obtained by BHCVRP and OR-Tools with the CMT heuristic in the HFVRP variant with Euclidean distance.

Instance	BHCVRP		OR-Tools	
	CMT		CH	
	Cost in distance	Execution time (s)	Cost in distance	Execution time (s)
HFVRP_1	**587.91 (4)**	0.61	2534.0 (3)	**0.14**
HFVRP_4	**802.23 (6)**	1.36	3753.0 (4)	**0.32**
HFVRP_p3	**848.13 (3)**	1.77	4085.0 (3)	**0.31**
HFVRP_p5	**1180.74 (3)**	3.41	1544.0 (3)	**0.56**
HFVRP_p14	**16782.7 (4)**	112.69	9220.0 (4)	**15.84**

The results in **Table 5** indicate that the SS heuristic performed better in three cases, while the SP heuristic stood out in the remaining two. It is worth noting that SV had the best execution times. Furthermore, **Table 6** shows that CMT yielded the best solutions, while CH had the best completion times.

Finally, has decided to conduct a final comparison by considering the best overall results for each component in the CVRP and HFVRP variants. Below, **Table 7** and **Table 8** summarize the results in terms of cost in Euclidean distance, total paths, execution time, and heuristics used. Once again, has carried out 20 runs for each instance with 20 evaluations.

Table 7. Summary of the overall results obtained by BHCVRP and OR-Tools in the five CVRP instances with Euclidean distance.

Instance	BHCVRP				OR-Tools			
	Heuristic	Cost in distance	Total paths	Execution time (s)	Heuristic	Cost in distance	Total paths	Execution time (s)
CVRP_1	SS	**558.89**	4	0.54	PCA	2152.00	4	**0.13**
CVRP_4	SP	**787.61**	6	1.25	FUMV	3916.00	6	**0.33**
CVRP_p3	MJ	**666.86**	3	15.75	ES	4133.00	3	**0.27**
CVRP_p5	SP	**814.49**	4	36.10	PCA	4534.00	4	**0.57**
CVRP_p14	SP	**6134.72**	4	94.80	FUMV	9298.00	4	**15.00**

Table 8. Summary of the overall results obtained by BHCVRP and OR-Tools in the five HFVRP instances with Euclidean distance.

Instance	BHCVRP				OR-Tools			
	Heuristic	Cost in distance	Total paths	Execution time (s)	Heuristic	Cost in distance	Total paths	Execution time (s)
HFVRP_1	SP	**474.33**	2	1.09	PCA	3225.00	4	**0.14**
HFVRP_4	MJ	**799.87**	6	9.36	PCA	3730.00	4	**0.30**
HFVRP_p3	SP	**653.83**	3	4.19	PCA	4030.00	3	**0.40**
HFVRP_p5	SS	**768.28**	3	2.66	PCA	1512.00	3	**0.71**
HFVRP_p14	SS	**6182.04**	4	24.61	SV	9327.00	4	**12.03**

In order to compare the results obtained by BHCVRP and OR-Tools in **Table 7** and **Table 8** more accurately, has decided to use the Wilcoxon nonparametric statistical test, as demonstrated in **Table 9** and **Table 10**. This test helps to determine if there are significant differences between the results produced by each software component.

Table 9. Wilcoxon test results of the comparison between BHCVRP and OR-Tools results in terms of cost in distance for CVRP and HFVRP variants.

Hypothesis	R+	R-	p-value	asymptotic p-value
CVRP variant				
BHCVRP vs OR-Tools	36.0	0.0	≥0.2	0.009583
OR-Tools vs **BHCVRP**	0.0	36.0	≥0.2	**1**
HFVRP variant				
BHCVRP vs OR-Tools	36.0	0.0	≥0.2	0.009583
OR-Tools vs **BHCVRP**	0.0	36.0	≥0.2	**1**

Table 10. Wilcoxon test results of the comparison between BHCVRP and OR-Tools results in terms of execution time for CVRP and HFVRP variants.

Hypothesis	R+	R-	p-value	asymptotic p-value
CVRP variant				
BHCVRP vs **OR-Tools**	0.0	36.0	≥0.2	**1**
OR-Tools vs BHCVRP	36.0	0.0	≥0.2	0.009583
HFVRP variant				
BHCVRP vs **OR-Tools**	0.0	36.0	≥0.2	**1**
OR-Tools vs BHCVRP	36.0	0.0	≥0.2	0.009583

The results from **Table 9** indicate significant differences in distance cost between the two software components, with BHCVRP providing significantly better solutions. Additionally, **Table 10** shows significant differences in execution time between the two software components, with OR-Tools providing the best solutions.

5 Conclusions

Based on the experiment conducted in this paper, it is suggested to use the parallel version of the heuristic Savings Algorithm (SP) to achieve better solutions for the Capacitated Vehicle Routing Problem (CVRP). For the Heterogeneous Fleet Vehicle Routing Problem (HFVRP), it is recommended to use the sequential version of the Savings Algorithm (SS). In addition, BHCVRP is shown to yield higher quality solutions, while OR-Tools is more suitable for situations where the primary goal is to obtain solutions in less execution time. Other interesting aspects to consider for future research are to increase the number of instances to test, to use distances such as Manhattan, Chebyshev, and Haversine, to explore other variants such as VRPTW and MDVRP, as well as to test with other classical heuristics in the literature such as Sweep Algorithm, Nearest Neighbor, among others.

References

[1] B. Golden, X. Wang, and E. Wasil, "The evolution of the Vehicle Routing Problem—A survey of VRP research and practice from 2005 to 2022," in *The Evolution of the Vehicle Routing Problem: A Survey of VRP Research and Practice from 2005 to 2022*: Springer, 2023, pp. 1-64.

[2] G. B. Dantzig and J. H. Ramser, "The truck dispatching problem," *Management science,* vol. 6, no. 1, pp. 80-91, 1959.

[3] G. Clarke and J. W. Wright, "Scheduling of vehicles from a central depot to a number of delivery points," *Operations research,* vol. 12, no. 4, pp. 568-581, 1964.

[4] M. Zirour, "Vehicle routing problem: models and solutions," *Journal of Quality Measurement and Analysis JQMA,* vol. 4, no. 1, pp. 205-218, 2008.

[5] G. D. Konstantakopoulos, S. P. Gayialis, and E. P. Kechagias, "Vehicle routing problem and related algorithms for logistics distribution: A literature review and classification," *Operational research,* vol. 22, no. 3, pp. 2033-2062, 2022.

[6] E. Arifta and F. Rakhmawati, "Analysis of Book Distribution Routes Using the Capacity Vehicle Routing Problem (CVRP) Method Using the Sweep Algorithm," *Sinkron: jurnal dan penelitian teknik informatika,* vol. 8, no. 1, pp. 360-367, 2023.

[7] F. Morsidi and I. Y. Panessai, "Overview of the Integral Impact of MDVRP Routing Variables on Routing Heuristics," *Applied Information Technology And Computer Science,* vol. 4, no. 1, pp. 1723-1738, 2023.

[8] V. R. Máximo, J.-F. Cordeau, and M. C. Nascimento, "An adaptive iterated local search heuristic for the Heterogeneous Fleet Vehicle Routing Problem," *Computers & Operations Research,* vol. 148, p. 105954, 2022.

[9] L. Accorsi and D. Vigo, "A hybrid metaheuristic for single truck and trailer routing problems," *Transportation Science,* vol. 54, no. 5, pp. 1351-1371, 2020.

[10] E.-G. Talbi, *Metaheuristics: from design to implementation*. Hoboken, New Jersey: John Wiley & Sons, Inc., 2009.

[11] F. Liu, C. Lu, L. Gui, Q. Zhang, X. Tong, and M. Yuan, "Heuristics for vehicle routing problem: A survey and recent advances," *arXiv preprint arXiv:2303.04147,* 2023.

[12] M. L. Fisher and R. Jaikumar, "A generalized assignment heuristic for vehicle routing," *Networks,* vol. 11, no. 2, pp. 109-124, 1981.

[13] M. H. Romanycia and F. J. Pelletier, "What is a heuristic?," *Computational intelligence,* vol. 1, no. 1, pp. 47-58, 1985.

[14] R. Martı, "Procedimientos metaheurısticos en optimización combinatoria," *Matemátiques, Universidad de Valencia,* vol. 1, no. 1, pp. 3-62, 2003.

[15] A. Olivera, "Heurísticas para problemas de ruteo de vehículos," *Reportes Técnicos 04-08,* 2004.

[16] A. C. Pérez-Pérez, E. Sánchez-Ansola, and A. Rosete-Suárez, "A metaheuristic solution for the school bus routing problem with homogeneus fleet and bus stop selection," *Ingeniería,* vol. 26, no. 2, pp. 233-253, 2021.

[17] Y. Koguma, "Tabu Search-based Heuristic Solver for General Integer Linear Programming Problems," *IEEE Access,* 2024.

[18] C. R. Gatica, S. M. Molina, and G. Leguizamón, "Evaluación de variantes de la metaheurística VNS para el problema de planificación de máquinas paralelas," in *XXVIII Congreso Argentino de Ciencias de la Computación (CACIC)(La Rioja, 3 al 6 de octubre de 2022),* 2023.

[19] C. Groër, B. Golden, and E. Wasil, "A library of local search heuristics for the vehicle routing problem," *Mathematical Programming Computation,* vol. 2, pp. 79-101, 2010.

[20] K. Florios and G. Mavrotas, "Generation of the exact pareto set in multi-objective traveling salesman and set covering problems," *Applied Mathematics and Computation,* vol. 237, pp. 1-19, 2014.

[21] L. Díaz-Subi, "Nueva versión de la Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos," Tesis de diploma, Facultad de Ingeniería Informática, Instituto Superior Politécnico José Antonio Echeverría (CUJAE), 2016.

[22] F. Didier, L. Perron, S. Mohajeri, S. A. Gay, T. Cuvelier, and V. Furnon, "OR-Tools' vehicle routing solver: a generic constraint-programming solver with heuristic search for routing problems," 2023.

[23] A. Mahmoud, T. Chouaki, S. Hörl, and J. Puchinger, "Extending JSprit to solve electric vehicle routing problems with recharging," *Procedia Computer Science,* vol. 201, pp. 289-295, March 22-25 2022.

[24] M. Karkula, J. Duda, and I. Skalna, "Comparison of capabilities of recent open-source tools for solving Capacitated Vehicle Routing Problems with Time Windows," *Carpathian Logistics Congress,* Accessed on: December 2-4, 2019

[25] M. PAREJO *et al.*, "Desarrollo de un algoritmo para seleccionar la ubicación de puntos de abastecimiento minimizando los tiempos y costos de transporte," *Revista Espacios,* vol. 41, no. 17, pp. 1-15, 2020.

[26] O. E. Calderón Calderón, "Diseño e implementación de modelo matemático para la optimización del recorrido de camiones recolectores de residuos sólidos en el municipio de León," 2020.

[27] M. Salehi Sarbijan and J. Behnamian, "Emerging research fields in vehicle routing problem: a short review," *Archives of Computational Methods in Engineering*, vol. 30, no. 4, pp. 2473-2491, 2023.

[28] J. M. Daza, J. R. Montoya, and F. Narducci, "Resolución del problema de enrutamiento de vehículos con limitaciones de capacidad utilizando un procedimiento metaheurístico de dos fases," *Revista EIA*, no. 12, pp. 23-38, 2009.

[29] A. P. Bravo Quispe, C. L. Yupanqui Aguilar, C. V. Cribillero Meza, D. A. Ocharan Rojas, and V. G. Patricio Miranda, "Gestión de rutas a través del uso de modelos basados en algoritmos," 2021.

[30] N. Pérez, "Incorporación de algoritmos de agrupamiento en la Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos," Tesis de diploma, Facultad de Ingeniería Informática, Universidad Tecnológica de La Habana José Antonio Echeverría (CUJAE), La Habana, 2022.

[31] T. Cuvelier, F. Didier, V. Furnon, S. Gay, S. Mohajeri, and L. Perron, "OR-Tools' Vehicle Routing Solver: a Generic Constraint-Programming Solver with Heuristic Search for Routing Problems," in *24e congrès annuel de la société française de recherche opérationnelle et d'aide à la décision*, 2023.

[32] E. Alba and A. J. Nebro, "Solving optimization problems with grid-enabled technologies," *ERCIM News*, vol. 59, p. 53, 2004.